SDS Exercise Sheet 5: Dialogue Policy

Nurul Lubis

Release 9.6.2026 — Due 29.6.2026

1 Introduction

In this exercise you are going to implement parts of the proximal policy optimization (PPO) algorithm and perform reinforcement learning (RL) with user simulator to train a dialogue policy.

The objective of this practical are as follows:

- Understand how dialogue performance is utilized for dialogue policy optimization with reinforcement learning (RL).
- Able to perform RL to optimize a dialogue policy.
- Able to analyse the result of RL training, drawing the relationship between reward signal and policy performance.

2 RL Toolkit in ConvLab3

2.1 Training Configuration

The **pipeline configuration** file defines the modules used in the pipeline, in a `json` dictionary format. The parameters related to the environment interactions are defined under the `model` key. The pipeline is then defined module by module. For each module, the class and initialization arguments need to be provided. The following starter pipeline config is provided
`convlab3/convlab/policy/ppo/configs/RuleUser-Semantic-RuleDST.json`

Additionally, a **training configuration** specifies hyperparameters that can be fine-tuned, which includes RL-related parameters such as discount factor and general parameters such as learning rate. The starter training configuration is provided under
`convlab3/convlab/policy/ppo/configs/ppo_config.json`

RL training may be performed at 2 levels of granularity. At semantic level, the RL would operate on dialogue actions, skipping the NLU and NLG in the pipeline. At word level, the RL would use the full pipeline, taking word sequence as input and outputting word sequence. Note that even when the full pipeline is used, in the default set up only the policy would be optimized during RL. In this exercise, RL will be performed at semantic level.

2.2 User Simulator

The RL training is done via interaction with user simulator (US) (covered in lecture 8), which serves as the environment, providing the state and state transitions as well as the reward signal. The US is an approximation of real users, which help reduce time and cost needed to train a dialogue policy. In this exercise, we will be using the rule-based US.

2.3 Evaluation and Result

Once RL training is completed, an experiment folder with a unique name will appear under `finished_experiment` folder. The experiment folder contains training log, evaluation result, as well as the policy model itself.

Additionally, the training produces tensorboard files, which plots various measures taken throughout training allowing easy monitoring and analysis. More information on tensorboard can be found here:

`https://www.tensorflow.org/tensorboard/get_started`

To combine multiple tensorboard files into one graph a plotting tool is also provided in

`convlab3/convlab/policy/plot_results/`

3 Exercises

3.1 Codebase

Use the same repository used in exercise 2. Make sure to **pull the latest code** before starting the exercise.

3.2 Tasks

The script used in tasks 1 and 2 is

`convlab3/convlab/policy/ppo/ppo.py`

Task 1: Advantage Estimation Function Implement the advantage estimation according to the equation proposed in [1]. You may also refer to the lecture slide for the equation. Implement your solution inside the box labeled `TODO Policy_TASK1`, from `1a` to `1c`.

Task 2: Policy Update Implement the policy update using PPO clip objective. Implement your solution inside the box labeled `TODO Policy_TASK2`, from `2a` to `2d`.

Task 3: Train Dialogue Policies Run dialogue policy training with RL. Train 2 versions of the dialogue policy by creating different configuration files. You are free to decide which hyperparameter(s) to tune. For example, would you like see the effect of different discount factors, trade off bias and variance in GAE, see the effect of the amount of data for training, or something else? Note that RL tend to be more sensitive to hyperparameters than supervised learning (SL), so the tuning may effect the final result significantly.

Run the model training by following the instruction in

`convlab3/convlab/policy/ppo/README.md`.

Tip: You may use the pre-trained policy from Exercise 2 as the initialization. Consult `ppo_config.json` and `RuleUser-Semantic-RuleDST.json`

Task 4: Incorporate the policy into a dialogue system pipeline Plug your policies from Task 3 above to the dialogue system pipeline from Exercise 2.

4 Result and Discussion

In your final report, write a short report covering, but not limited to, the following points. Additional insights, analysis, and discussion are highly encouraged.

1. Compare and analyze the dialogue policies you have trained. You may look at the tensorboard graph generated during training, for example at `success rate` and `avg_actions`.
2. Interact with your dialogue policies using the pipeline from Task 4. Provide an example dialogue from this interaction. How does it compare to the policy from Exercise 2?
3. Does the interaction quality align with the performance measured during RL? Analyze and explain the results, i.e. what is happening and why does it happen?

Push your configs and completed scripts into your git repository and put the link to your repository in your report.

References

[1] Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438* (2015).